

Resenha - Criteria for Modularization - D.L. Parnas

Neste artigo, D.L. Parnas discute como a forma como dividimos um sistema em módulos afeta drasticamente sua flexibilidade, compreensão e tempo de desenvolvimento. Parnas aponta que a eficiência da modularização não está relacionado apenas na divisão em si, mas nos critérios utilizados para essa separação.

Critérios de Decomposição

Parnas compara duas abordagens principais utilizando o exemplo de um sistema de indexação KWIC (Key Word In Context):

1. Decomposição Convencional (Baseada em Fluxograma):

Cada módulo representa um passo no processamento (Entrada, Processamento, Saída). Parnas critica essa visão, pois ela torna o sistema rígido: mudanças no formato de armazenamento de dados, por exemplo, obrigariam alterações em quase todos os módulos normalmente, uma vez que todos compartilham o conhecimento da estrutura interna.

2. Ocultamento de Informações (Information Hiding):

Esta é a proposta central de Parnas. Cada módulo é projetado para esconder uma decisão de design difícil ou provável de mudar. O conhecimento sobre como os dados são armazenados ou como um algoritmo funciona fica restrito ao módulo. Os outros módulos interagem apenas por meio de interfaces abstratas.

Pontos de análise

Independência: O "Information Hiding" permite que equipes trabalhem de forma independente muito mais cedo, pois as interfaces são mais simples e abstratas.

Manutenibilidade: Mudanças drásticas podem ser feitas em um módulo sem afetar os outros.

Eficiência: Parnas mostra que a segunda abordagem pode ser menos eficiente se implementada de forma ingênua (devido a muitas chamadas de função), sugerindo que o código final pode precisar de técnicas de montagem ou compilação que eliminem esse custo extra.

Exemplos Práticos

1. Modularização Convencional (Baseada em Tarefas/Fluxo)

O código é dividido pelo que ele faz sequencialmente.

Módulo Leitor: Abre o arquivo 'tarefa.json' e carrega o array global de strings.

Módulo Adicionador: Pega o array global, adiciona uma nova string e valida o tamanho da string.

Módulo Gravador: Pega o array global e sobrescreve o arquivo tasks.json.

Problema: Se a gente decidir mudar de um arquivo JSON para um Banco de Dados SQL, terá que reescrever todos os três módulos, já que todos manipulam diretamente o array ou conhecem a estrutura do arquivo.

2. Modularização de Parnas (Information Hiding)

Esconde a decisão de como os dados são salvos.

Módulo de Armazenamento (Data Hiding): Contém funções como salvarTarefa(texto) e obterTodas(). Só ele sabe se está usando json, SQL ou memória.

Módulo de Interface de Usuário: Apenas chama salvarTarefa(). Sem saber onde ou como a tarefa é guardada.

Módulo de Regras de Negócio: Verifica se o texto é válido e envia para o Módulo de Armazenamento.

Vantagem: Se você mudar de json para SQL, alteramos apenas o Módulo de Armazenamento. A Interface e as Regras de Negócio continuam funcionando sem alterações.